# Contents

**Software Design Document — music-room** **1**
1. Introduction . . . 1
2. References . . . 2
3. Stakeholders and design concerns . . . 2
4. Context viewpoint . . . 3
5. Composition viewpoint . . . 3
6. Logical viewpoint — domain model . . . 4
7. Dependency viewpoint . . . 5
8. Information viewpoint . . . 6
9. Interface viewpoint . . . 7
10. Interaction viewpoint . . . 8
11. State dynamics . . . 10
12. Concurrency viewpoint . . . 10
13. Security design . . . 11
14. Deployment and resource viewpoint . . . 14
15. Design rationale . . . 15
16. Requirements traceability . . . 16
17. Open design issues . . . 23
Related . . . 26

# Software Design Document — music-room

Consolidated design view of the system, organised by the viewpoints of IEEE 1016.

**Status: partly implemented.** Slices S0 to S2 are built — the identity surface, the credential rate limits and the audit log — on stacked draft pull requests; §16 says which rows that covers and which are still design only. Every section below describes what is decided, and marks what is not with `UNKNOWN - requires decision`. A guess is never substituted for a decision.

---

## 1. Introduction

### 1.1 Purpose

One document an evaluator, a new teammate, or a future maintainer can read to understand how the system is designed and why. It consolidates the design; it does not replace the layers that own each part.

### 1.2 Scope

The `music-room` backend and the contract it exposes to clients. The React Native client is out of the current work scope — only the constraints it places on the backend appear here ([[ADR-0002-mobile-react-native]]).

### 1.3 Audience

Peer evaluators, team members, and any agent working in this repository.

### 1.4 Authority order

When two sources disagree:

```
42 subject  ->  ADR  ->  standard  ->  this document  ->  note  ->  code comment
```

This document **consolidates**; it is not the source of truth for anything. Rationale lives in ADRs, enforceable rules live in `docs/technical-guide/`, requirements live in the subject.

### 1.5 Relationship to the other layers

| Layer | Owns | This document does |
|---|---|---|
| `brain/constraints/` | what is required | quote and trace it |
| `brain/decisions/` | why a choice was made | reference it, never restate the rationale |
| `brain/architecture/` | the working design notes | consolidate them into one readable view |
| `docs/technical-guide/` | enforceable rules | reference, never copy |
| `docs/owasp/` | the security conformance target and its evidence | reference, never copy |
| `MusicRoom/` | what the code does | describe intent only |

---

## 2. References

| Ref | Source |
|---|---|
| R1 | 42 subject "Music Room", v6 — `docs/subject/en.subject tr.pdf`, distilled in [[subject-requirements]] |
| R2 | [[decisions-index]] — ADR-0001 through ADR-0006 |
| R3 | [[technical-guide-index]] — 19 engineering standards |
| R4 | IEEE 1016 — software design description, used here as the section structure |
| R5 | RFC 9110 (HTTP semantics), RFC 9457 (Problem Details), RFC 9700 (OAuth 2.0 BCP), OpenID Connect Core 1.0 |
| R6 | OWASP ASVS 5.0.0, vendored at `docs/owasp/upstream/asvs-5.0.0/`; conformance scope in [[asvs-conformance]] |
| R7 | NIST SP 800-63B — password and verifier requirements |

---

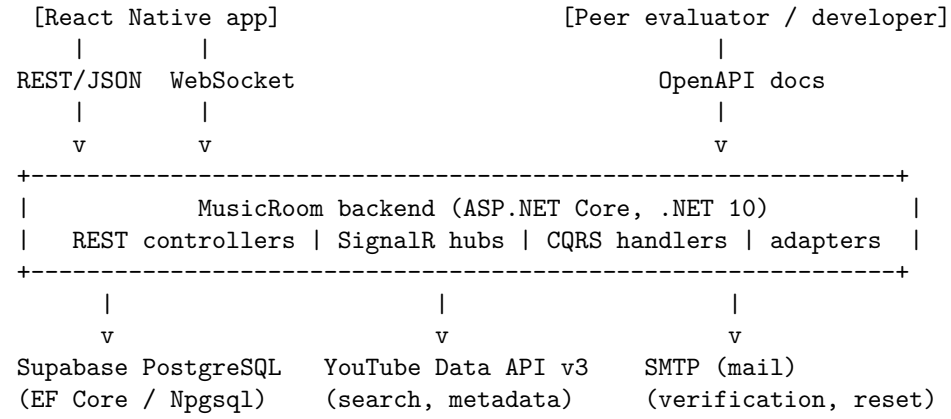
## 3. Stakeholders and design concerns

| Stakeholder | Primary concern | Addressed in |
|---|---|---|
| End user at an event | the queue reflects the group's votes, immediately | §11, §12 |
| Event owner | control over who may vote and edit | §9.2, §13.2 |
| Peer evaluator | every mandatory requirement is met and defensible | §16 |
| Team developer | knowing where code goes and which rule applies | §5, §7 |
| Operator | the system can be run, measured, and recovered | §14 |

---

## 4. Context viewpoint

### 4.1 System context




### 4.2 External actors and interfaces

| External element | Direction | Protocol | Owned by |
|---|---|---|---|
| React Native client | inbound | HTTPS REST/JSON, WebSocket | us (deferred) |
| Supabase PostgreSQL | outbound | PostgreSQL wire protocol | vendor, managed |
| YouTube Data API v3 | outbound | HTTPS REST | vendor |
| YouTube player SDK | client-side only | — | vendor |
| Google / Facebook OIDC | outbound | OAuth 2.0 + PKCE | vendor |
| Mail provider | outbound | HTTPS API | hosted free tier – [[ADR-0013-mail-provider]] |

**Design constraint.** No client ever reaches Supabase or YouTube Data API directly. Subject R1 V.3 makes the backend the single guardian of truth, and V.6 makes it the single authorisation gate. A direct client connection would route around both. See [[ADR-0006-datastore]].

---

## 5. Composition viewpoint

### 5.1 Physical composition

One repository, two application roots – [[ADR-0020-monorepo-layout]]:

| Root | Contents | Owner |
|---|---|---|
| `MusicRoom/` | backend, .NET 10 | backend developer |
| `MusicRoomApp/` | React Native client | UI developer |

One clone is the whole submission, which subject VII requires. Neither root writes into the other; the contract between them is [[endpoint-inventory]] and the generated `docs/api/openapi.json`. `reference/` is read-only study material, never built, referenced or copied from – see [[reference-narchitecture]].

Five projects, per [[clean-architecture-cqrs]]:

| Project | Responsibility |
|---|---|
| `Domain` | entities, value objects, enums, domain rules, domain exceptions |
| `Application` | commands, queries, handlers, validators, pipeline behaviours, abstractions |
| `Persistence` | `DbContext`, entity configurations, repositories, EF Core migrations |
| `Infrastructure` | YouTube adapter, mail, token and OTP services |
| `WebAPI` | controllers, SignalR hubs, middleware, DI composition, OpenAPI |

Layout mirrors the reference repository – [[ADR-0011-solution-layout]]:

```
MusicRoom/
  MusicRoom.sln
  src/
    corePackages/   Core.Application | Core.CrossCuttingConcerns | Core.Persistence | Core.Security
    MusicRoom/      Domain | Application | Persistence | Infrastructure | WebAPI
  tests/
    <Project>.Tests/
```

The shape is copied; no file is. A type enters `corePackages` only if it is domain-agnostic.

### 5.2 Logical composition — bounded contexts

| Context | Owns | Subject ref |
|---|---|---|
| Identity | user, credentials, email verification, password reset, social links | V.1 |
| Profile | profile fields with per-field visibility, music preferences | V.1 |
| Event (Track Vote) | event, attendee, track suggestion, vote, licence policy | V.2.1 |
| Playlist (Editor) | playlist, collaborator, ordered tracks | V.2.3 |
| Delegation (Control) | device, control grant | V.2.2 |
| Playback | playback session, now-playing pointer, queue advance | V.2.1, V.2.2 |
| Catalog | YouTube track lookup and cache | V.2, III |
| Audit | per-action record: platform, device, app version | V.6 |

Subject R1 V.2 requires at least 2 of 3 services. **All three ship** – [[ADR-0010-service-scope]]. Identity, Profile, Catalog and Audit are required regardless.

Consequence: the location+time licence mode is mandatory, decided in [[ADR-0014-location-time-licence]]. If the schedule slips, Control Delegation is dropped first – it is the smallest, and removing it still leaves two services, satisfying V.2.

---

## 6. Logical viewpoint — domain model

Detail source: [[domain-model]]. Field-level detail is deliberately absent until the first migration exists.

### 6.1 Aggregates

| Aggregate | Root | Key invariant |
|---|---|---|
| User | `User` | email unique; an unverified email cannot use services |
| Profile | `Profile` | every field carries one of `Public / Friends / Private` |
| Event | `Event` | one vote per user per track; ranking derives from vote count |
| Playlist | `Playlist` | position ordering stays total and gap-tolerant |
| Device | `Device` | a control grant belongs to exactly one device |
| PlaybackSession | `PlaybackSession` | at most one active session per device; `controllerUserId` is what a control grant changes |
| TrackRef | value object | a YouTube reference only — never media content |

### 6.2 Shared enumerations

| Enum | Values | Source |
|---|---|---|
| `Visibility` | `Public`, `Private` | R1 V.2.1, V.2.3 |
| `LicenceMode` | `Everyone`, `InvitedOnly`, `LocationAndTimeRestricted` | R1 V.2.1, V.2.3 |
| `PresenceProof` | `Geofence`, `EventCode`, `Both` | policy carried by `LocationAndTimeRestricted` – [[ADR-0014-location-time-licence]] |
| `ProfileFieldVisibility` | `Public`, `Friends`, `Private` | R1 V.1 |
| `Platform` | `iOS`, `Android`, `Web`, `Other` | R1 V.6 |

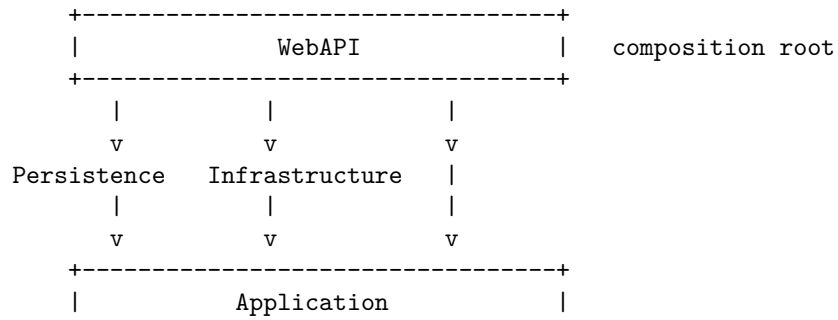
### 6.3 Deliberate exclusions

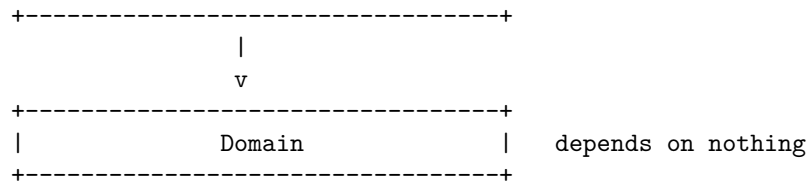
Audio and video content is never stored, cached, proxied or re-hosted. Only `TrackRef` metadata is persisted — [[ADR-0003-music-provider-youtube]].

---

## 7. Dependency viewpoint

### 7.1 The dependency rule






Enforced constraints, from [[clean-architecture-cqrs]]:

- `Domain` references no other project and no infrastructure package.
- `Application` references neither EF Core, nor ASP.NET Core, nor any vendor SDK.
- `WebAPI` never touches `DbContext` directly.
- A reference pointing outward is a build-breaking defect, not a style issue.

### 7.2 External dependency boundaries

| External system | Isolated behind | Located in |
|---|---|---|
| YouTube Data API | one adapter, interface declared in `Application` | `Infrastructure` |
| SMTP | mail service abstraction | `Infrastructure` |
| Google / Facebook | provider token validation service | `Infrastructure` |
| PostgreSQL | repositories + `DbContext` | `Persistence` |

---

## 8. Information viewpoint

### 8.1 Storage

Supabase-managed PostgreSQL, accessed through EF Core with the Npgsql provider ([[ADR-0006-datastore]]). Only the Postgres surface is used — Supabase Auth, Realtime, Storage and PostgREST are deliberately not adopted, because identity, realtime and the API are ours.

### 8.2 Constraints that carry invariants

Conventions and full rules: [[database-design-migrations]].

| Constraint | Protects | Why it is in the database |
|---|---|---|
| `ux_votes_event_id_track_id_user_id` | one vote per user per track | a read-then-write check in C# is a race condition |
| `ux_users_email` | account uniqueness | concurrent registration |
| `xmin` concurrency token on `playlists` | reorder conflicts | last-writer-wins must be detectable |
| `xmin` concurrency token on `devices` | control takeover | two simultaneous claims must not both succeed |

### 8.3 Common columns

Every table: `id`, `created_at_utc`, `updated_at_utc`. User-owned aggregates use soft delete (`deleted_at_utc`) so the audit trail survives. All instants are `timestamp with time zone`, written as UTC.

### 8.4 Ordering strategy

Playlist position uses sparse ordering keys with gaps, so inserting between two tracks does not rewrite the list. See §12.2.

### 8.5 Schema ownership and evolution

- EF Core migrations are the only mechanism that changes the schema. Editing the schema in the Supabase dashboard desynchronises migration history and is treated as an incident.
- Two connection strings: pooled (Supavisor) for the API, **direct** for migrations.
- Destructive changes follow expand -> migrate -> contract.
- Every migration is recorded in [[migrations-index]].

Concrete table and column definitions live in the migrations and their `IEntityTypeConfiguration` files, which are authoritative — four migrations so far, `users` through `audit_records` ([[migrations-index]]). For a table no migration has created yet it is still `UNKNOWN - requires decision`.

---

## 9. Interface viewpoint

### 9.1 REST API

Rules: [[rest-api-design]]. Documentation: [[openapi-documentation]].

| Aspect | Design |
|---|---|
| Base path | `/api/v1` |
| Format | JSON, `camelCase`, UTF-8 |
| Errors | RFC 9457 `application/problem+json`, one shape, with `traceId` |
| Collections | envelope `{ items, page, pageSize, total }`, page size capped |
| Timestamps | UTC ISO-8601 |
| Privacy rule | a resource hidden from the caller returns `404`, never `403` |
| Auth | bearer JWT |
| Mandatory headers | `X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version` on mutations |
| Documentation | OpenAPI 3.1 generated from code, exported to `docs/api/openapi.json` |

The endpoint inventory is [[endpoint-inventory]] – around 60 endpoints across Identity, Profile, Catalog, Track Vote, Playlist Editor, Control Delegation, audit and operations, plus the client-to-server hub methods. Concrete request and response schemas come from the generated OpenAPI document, not from that file.

### 9.2 Realtime API

Rules: [[signalr-realtime]]. Rationale: [[ADR-0004-realtime-signalr]].

| Hub | Group key | Server-to-client events |
|---|---|---|
| `EventHub` | `event:{eventId}` | `TrackSuggested`, `VoteChanged`, `QueueReordered`, `NowPlayingChanged` |
| `PlaylistHub` | `playlist:{playlistId}` | `TrackAdded`, `TrackRemoved`, `TrackMoved`, `CollaboratorChanged` |
| `ControlHub` | `user:{userId}` | `ControlGranted`, `ControlRevoked`, `ControlTakenOver` |

Contract rules:

- Hubs are transport only. Every hub method dispatches the **same** CQRS handler the REST endpoint uses, so the two transports cannot diverge.
- Every payload carries the aggregate id and a monotonic `version`, so a client that missed a message can detect the gap and resynchronise over REST.
- Group membership is authorised server-side on every join and re-checked on reconnect.
- A broadcast is emitted only after the transaction commits.

### 9.3 External interfaces

| Interface | Usage | Design note |
|---|---|---|
| YouTube Data API v3 | `videos.list` preferred; `search.list` only on real user input | quota is billed in units and `search.list` is the expensive call; results are cached as normalised `TrackRef` |
| YouTube player SDK | client-side playback | backend decides what and when, never streams media |
| Google / Facebook | authorization code flow with PKCE | provider token validated server-side against JWKS; a client-supplied profile is never trusted |
| Mail provider | verification and reset mail | hosted free tier behind `IMailService`; API key server-side – [[ADR-0013-mail-provider]] |

---

## 10. Interaction viewpoint

Representative flows. Steps are design intent, not implemented code.

### 10.1 Registration with email and password

```
1. Client POST /api/v1/auth/registrations
2. Validation behaviour rejects malformed input, including the 12-64 character
   and 72-byte UTF-8 password limits that BCrypt requires
3. Handler hashes the password with BCrypt at cost >= 12, persists an unverified User
4. Infrastructure sends a verification mail with a single-use, hashed, 24h token
5. Response 202, always -- identical whether or not the address is already registered.
   A 409 here would be an account-enumeration oracle; the mailbox tells the real story.
   The account cannot use services until verified
6. Client POST /api/v1/auth/email-verifications with the token
7. Handler marks the account verified and consumes the token
```

Algorithm: [[ADR-0007-password-hashing-bcrypt]]. Paths and status codes: [[endpoint-inventory]] §1.

### 10.2 Social login

OpenID Connect, not bare OAuth 2.0 – [[ADR-0008-oidc-social-login]].

```
1. Device runs the OIDC authorization code flow with PKCE
2. Client POST /api/v1/auth/social with the provider ID TOKEN (never an access token)
3. Server validates, in order:
```

```
     a. signature against the provider JWKS from its discovery document
     b. iss matches the expected issuer
     c. aud matches our client id      <- stops token substitution
     d. exp and iat within tolerance
     e. nonce matches the one this server generated
     f. email_verified is true before any email-based account match
4. Server maps (provider, providerUserId) to a local user, or creates one
5. Server issues ITS OWN access + refresh token pair
   - a provider token is never accepted as an API credential
```

Provider asymmetry: Google is a full OIDC provider; Facebook is not uniformly OIDC (Limited Login issues an ID token, classic Facebook Login does not). The integration branches. See [[social-oauth]].

### 10.3 Casting a vote — the contended write path

```
1. Client POST /api/v1/events/{eventId}/tracks/{trackId}/votes
2. Authorisation behaviour evaluates ownership, then Visibility and LicenceMode
3. Rate limiting behaviour applies per account and per IP
4. Transaction behaviour opens the transaction
5. Handler inserts the vote row
   - the unique index is the arbiter; a duplicate insert fails here, not in C#
   - a duplicate is mapped to 409 Conflict, never silently ignored
6. Vote count is recomputed from the source rows, never incremented in memory
7. Transaction commits
8. ONLY THEN: EventHub broadcasts VoteChanged and QueueReordered to event:{eventId}
```

Step 8 after step 7 is the rule that prevents clients from seeing state the server rolls back.

### 10.4 Moving a track in a playlist

```
1. Client PATCH .../playlists/{id}/tracks/{trackId} with the target position,
   carrying the playlist's current concurrency token
2. Authorisation behaviour evaluates Visibility and LicenceMode for edit rights
3. Handler computes a new sparse ordering key between the neighbours
4. Save fails with DbUpdateConcurrencyException if another edit landed first
   - the handler reloads and reapplies, or returns 409
5. On commit, PlaylistHub broadcasts TrackMoved with the new version
6. Clients treat the event as a reconcile hint, tolerating duplicates and reordering
```

### 10.5 Delegating music control

```
1. Owner POST .../devices/{deviceId}/control-grants with the target friend
2. Handler transitions the Device control state under its concurrency token
3. A competing claim loses on the concurrency token - it must not silently win
4. ControlHub notifies user:{ownerId} and user:{granteeId}
```

### 10.6 Advancing the queue

```
1. Controller device detects the current track ended
2. POST /api/v1/playback-sessions/{id}/advances  { expectedNowPlayingId }
3. Authorisation behaviour verifies the caller is the session's controllerUserId
4. Handler picks the next item:
     Event    -> highest vote count, ties broken by earliest suggestion
     Playlist -> next ordering key
5. Write under the concurrency token; a stale expectedNowPlayingId returns the current
   state instead of skipping a second track
```

```
6. Commit
7. ONLY THEN: NowPlayingChanged broadcast to the source's group
```

The server is the arbiter of **what** plays; the client is the authority on **when** it ended. The backend cannot observe audio, and a `durationSeconds` timer drifts on pause and buffering. See [[ADR-0017-playback-session]].

### 10.7 Every mutating request — the mandatory audit path

```
1. Middleware reads X-Client-Platform, X-Client-Device, X-Client-App-Version
2. Missing headers on a mutating request -> 400 (a silent default would void R1 V.6)
3. Values are attached to the request scope
4. An audit behaviour writes one audit record per handled command
```

---

## 11. State dynamics

### 11.1 Account state

```
Registered(unverified) --verify--> Active
Active --reset password--> Active (all refresh tokens revoked)
```

An unverified account cannot use the three services.

### 11.2 Playback session state

```
None --create--> Playing(nowPlaying)
Playing --advance--> Playing(next)      server picks next; idempotent on expectedNowPlayingId
Playing --advance--> Ended              queue exhausted
Playing --delete--> None
```

If the controller device goes silent mid-track, nothing advances. Accepted: control can be taken over via `PUT /devices/{id}/control` and the owner can always revoke. No server-side watchdog – it would reintroduce the drift this design avoids.

### 11.3 Device control state

Single-row state machine on `Device`, guarded by a concurrency token:

```
Uncontrolled --claim--> ControlledBy(owner)
ControlledBy(owner) --delegate--> ControlledBy(grantee)
ControlledBy(x) --revoke--> ControlledBy(owner)
ControlledBy(x) --release--> Uncontrolled
```

Concurrent transitions are resolved by the token. Exact state names are settled when the Delegation slice is built.

---

## 12. Concurrency viewpoint

Subject R1 V.2.1 and V.2.3 both call out race conditions explicitly. They are graded, so they are designed here rather than discovered in review. Full notes: [[realtime-architecture]].

### 12.1 Hotspots

| # | Hotspot | Failure if unhandled |
|---|---|---|
| 1 | two users vote on the same track simultaneously | duplicate votes, wrong ranking |
| 2 | two users reorder the same playlist simultaneously | divergent order, lost move |
| 3 | two devices claim control at once | two controllers, undefined playback |
| 4 | two clients advance the same playback session at once | a track is silently skipped |

### 12.2 Mechanisms

| Hotspot | Mechanism |
|---|---|
| 1 | unique index `(EventId, TrackId, UserId)`; duplicate detected by the database, mapped to `409` |
| 1 | vote counts recomputed inside the transaction or aggregated database-side — never read-modify-write |
| 2 | sparse ordering keys plus an `xmin` concurrency token; on conflict, reload and reapply, else `409` |
| 3 | single-row state machine with a concurrency token; the losing writer must lose visibly |
| 4 | `expectedNowPlayingId` in the request plus the session's concurrency token; a stale expectation returns current state rather than advancing again |

### 12.3 Governing principles

1. **The database is the arbiter, not the hub and not the application.**
2. **Broadcast after commit.** Never inside the transaction.
3. **Every realtime event is a reconcile hint.** Clients must tolerate duplicates and reordering.
4. **A sequential test proves nothing about a race.** Concurrency cases run in parallel and assert final database state — [[testing]].

---

## 13. Security design

Full rules: [[authentication-authorization]], [[application-security]], [[secrets-management]]. Design notes: [[auth-and-identity]]. Conformance target: [[asvs-conformance]] — ASVS 5.0.0 Level 1 for every applicable chapter, Level 2 for V6 Authentication, V7 Session Management, V8 Authorization, V9 Self-contained Tokens, V10 OAuth and OIDC, and V16 Security Logging and Error Handling.

### 13.1 Authentication

| Element | Design |
|---|---|
| Password hashing | **BCrypt** (`BCrypt.Net-Next`), work factor >= 12, rehash on login when the stored cost is lower – [[ADR-0007-password-hashing-bcrypt]] |
| Password policy | 12 to 64 characters, **<= 72 bytes UTF-8** (BCrypt truncates beyond that), no composition rules, no forced rotation (NIST SP 800-63B) |

| Element | Design |
|---|---|
| Session | 15-minute **RS256** JWT access token + opaque, hashed, rotating refresh token. The refresh family's lifetime is **30 days, absolute** — every rotation inherits the family's original expiry, so a stolen family cannot be kept alive by rotating it (owner decision, 2026-09-27, SPEC-005). From S8 an **ID token** joins the access token and both are issued by the API's own OpenID Provider surface – [[ADR-0031-musicroom-as-openid-provider]] |
| Theft detection | reuse of a rotated refresh token revokes the whole family |
| Verification / reset | single-use, hashed, short-TTL tokens; identical responses whether or not the account exists |
| Social | **OpenID Connect**; one validation core driven by each provider's discovery document, acquisition branching only: Google authorization code + PKCE `S256`, Facebook Limited Login. The ID token is validated server-side including `aud` and `nonce`; one provider identity maps to at most one user – [[ADR-0008-oidc-social-login]] |
| Provider role | from S8 the API is itself an **OpenID Provider**: discovery document, JWKS, authorization endpoint with PKCE `S256`, token endpoint, ID token, UserInfo. No resource-owner password-credentials grant – [[ADR-0031-musicroom-as-openid-provider]] |

### 13.2 Authorisation — two independent gates

Both are mandatory, both server-side, both implemented as a pipeline behaviour rather than scattered conditionals:

1. **Ownership** — the caller owns the resource or is an authorised participant (R1 V.6).
2. **Visibility and licence** — resolved from `Visibility` and `LicenceMode`, evaluated on every request and every hub join.

Profile responses are filtered **per field** by the requester's relationship (self / friend / stranger). A full row is never returned for the client to filter.

Row Level Security is enabled in PostgreSQL with deny-by-default policies as defence in depth. It is never the only gate.

### 13.3 Abuse resistance

Rate limits keyed on account **and** IP for login, registration, reset, verification resend and vote casting. Progressive backoff on repeated login failure. Generic error text to prevent account enumeration. `429` with `Retry-After`.

The numbers, approved by the repository owner on 2026-09-27 (SPEC-006). Implemented values, not intent: the per-IP half is the built-in middleware, one policy per endpoint; the per-account half is `ICredentialAttemptLimiter`, enforced inside the handler because the address a request names is known only after model binding. Both halves hold state **in process**, which is correct for the single instance [[ADR-0015-hosting-and-deployment]] deploys and would not be shared across several – O8.

| Endpoint | Per IP | Per named account |
|---|---|---|
| `POST /auth/sessions` | 20 / 5 min | 5 failures / 5 min, then a penalty of 1 window doubling to a ceiling of 8 |
| `POST /auth/tokens` | 60 / 5 min | – the request names no account, only a token |
| `POST /auth/registrations` | 10 / hour | the same per-account counter, charged on **every** attempt |
| `POST /auth/verification-requests` | 10 / hour | the same, charged on every attempt – each one writes a row and spends mail quota (T-19) |
| `POST /auth/email-verifications` | 30 / 5 min | – the request names no account |
| `POST /auth/password-reset-requests` | 10 / hour | the same per-account counter, charged on **every** attempt – SPEC-007 |
| `POST /auth/password-resets` | 30 / 5 min | – the request names no account, only a token |

Login charges only a *failed* attempt and clears the counter on success. Registration and resend charge every attempt, because the cost is in the attempt rather than in its outcome. The defaults are compiled in and overridable from configuration; a non-positive override is ignored, so configuration cannot switch a limit off.

### 13.4 Audit log — mandatory, and distinct from application logging

| | Audit log | Application log |
|---|---|---|
| Purpose | satisfy R1 V.6 — record what a user did | diagnose the system |
| Store | database table, queryable | structured log sink |
| Retention | kept; it is evidence | rotated |

Audit record: `userId`, `action`, `targetType`, `targetId`, `platform`, `device`, `appVersion`, `occurredAtUtc`, `correlationId`, outcome. See §10.7.

**Both transports are covered.** REST commands go through the pipeline behaviour; SignalR `JoinEvent`, `LeaveEvent`, `JoinPlaylist` and `LeavePlaylist` go through a hub filter writing the same record – [[ADR-0019-hub-action-auditing]]. Client context is captured at the handshake, since a hub invocation carries no HTTP headers, and a handshake without it is rejected. Connect and disconnect are deliberately not audited: mobile reconnect noise would bury deliberate user actions.

### 13.5 Never logged, never stored, never shipped to a client

Passwords, tokens, refresh tokens, API keys, `Authorization` headers, database connection strings, Supabase keys, OAuth client secrets. A mobile bundle can be unpacked, so it holds none of these.

### 13.6 Residual risk

R1 V.6 requires the remaining threats to be identified and their protections described. The register is [[threat-register]]: 18 threats with likelihood, impact, planned mitigation, residual risk, accepted-by and verification, plus an explicit out-of-scope list.

Every unfixed finding from [[owasp-top10-checklist]] or [[owasp-api-top10-checklist]] gets a row there before the pull request merges. An accepted risk that is written down is a defensible engineering decision; an unwritten one is an oversight.

---

## 14. Deployment and resource viewpoint

### 14.1 Topology

| Element | Where | Note |
|---|---|---|
| API | local Docker on a declared machine – [[ADR-0015-hosting-and-deployment]] | multi-stage image, pinned base, non-root user |
| Database | S0-S3 a **local PostgreSQL container**; S4 onward Supabase Free tier, Frankfurt – [[ADR-0026-environment-bring-up]] | same engine, same migrations, different connection string |
| Client | user devices | React Native, built by the second team member after the backend slices land |
| CI | GitHub Actions, separate `dev` and `prod` configurations, plus an independent client job | builds and tests; **does not deploy** |
| Observability | Serilog to stdout; OpenTelemetry exporter configuration-driven, **off by default** – [[ADR-0022-observability-stack]] | switched on for a load run |
| Secrets | `EnvSecretProvider` for the everyday loop; `InfisicalSecretProvider` behind the same abstraction, vault self-hosted in the compose stack under a `vault` profile – [[ADR-0029-containerised-dev-and-secret-vault]] | **required scope.** The abstraction lands early, the vault at S1b, and the delivered system runs on it |

The defence runs on the local stack. No remote deployment target is chosen yet; Vercel cannot host this API (serverless, no long-lived process, no persistent WebSocket).

The database being off-premise must be stated in the load report: network latency to Supabase is part of the measured numbers, not an excuse for them.

### 14.2 Reproducibility

Subject R1 IV.1 requires dependencies to install automatically from a fresh clone. The path:

```
clone  ->  cp .env.example .env  ->  fill values  ->  make up
```

`Makefile` and `docker-compose.yml` exist, and the path above is verified end to end: the API container serves `/health` and `/health/ready` against the compose PostgreSQL. Built in slice S0 of [[delivery-plan]]; rules in [[ci-cd-containerization]]. Issue O10 is closed. CI builds and tests but does not deploy – [[ADR-0015-hosting-and-deployment]].

### 14.3 Configuration

Development is **fully containerised**: `docker compose up` brings up the stack and the host needs Docker and nothing else – [[ADR-0029-containerised-dev-and-secret-vault]]. No documented step requires a locally installed SDK, database client or CLI; `dotnet-ef` is pinned through `.config/dotnet-tools.json`.

Secrets are read through a provider abstraction, so the vault replaces the environment without touching a call site. The vault is **required scope** – [[ADR-0029-containerised-dev-and-secret-vault]], amended 2026-09-27 – and it lands at S1b. It **shrinks** `.env` to the vault's own bootstrap credentials; it does not remove it, and subject V.8 still applies to what remains.

All configuration from environment variables, named after the role rather than the vendor (`DATABASE_URL`, not a provider name) – [[ADR-0026-environment-bring-up]]. External credentials are created just in time: mail before S1, YouTube and the Supabase project before S4, Google and Facebook before S8. Every key is listed in `.env.example` with a placeholder. `.env` is git-ignored. R1 V.8 makes a publicly stored credential an automatic project failure.

### 14.4 Capacity

Tool: **Gatling**, from its Docker image, simulations committed under `tests/load/` – [[ADR-0018-load-testing-tool]]. It is named in the subject and supports WebSocket natively, so one methodology covers both REST throughput and SignalR fan-out.

Subject R1 V.7 requires a measured and justified concurrent-user figure with the server specification declared. The report at `docs/load/REPORT.md` does not exist yet. REST throughput and SignalR connection fan-out are measured **separately** — they are different limits. See [[load-testing]].

The API is on-premise and the database is off-premise: latency to Frankfurt is inside every measured number, and the report must say so rather than gloss over it. Confirm the free tier's current connection ceiling and quotas on the day of the run – the connection ceiling, not storage, is the operative limit.

---

## 15. Design rationale

Rationale is not restated here. Each decision has its own record with the alternatives that were rejected and why — evaluators ask "why not X", and the answer must be on file.

| ADR | Decision | Status |
|---|---|---|
| [[ADR-0001-backend-dotnet-narchitecture]] | .NET 10, Clean Architecture + CQRS | Accepted |
| [[ADR-0002-mobile-react-native]] | React Native client, deferred | Accepted |
| [[ADR-0003-music-provider-youtube]] | YouTube as the music provider | Accepted |
| [[ADR-0004-realtime-signalr]] | SignalR over WebSocket | Accepted |
| [[ADR-0005-api-style-rest-json]] | REST + JSON + OpenAPI | Accepted |
| [[ADR-0006-datastore]] | Supabase managed PostgreSQL | Accepted |
| [[ADR-0007-password-hashing-bcrypt]] | BCrypt for password hashing | Accepted |
| [[ADR-0008-oidc-social-login]] | OpenID Connect for Google and Facebook | Accepted |
| [[ADR-0009-code-quality-gates]] | OWASP conformance and enforced code-size limits | Accepted |
| [[ADR-0010-service-scope]] | Ship all three services | Accepted |
| [[ADR-0011-solution-layout]] | Layout mirrors nArchitecture | Accepted |
| [[ADR-0012-test-stack]] | xUnit, NSubstitute, Testcontainers | Accepted |

| ADR | Decision | Status |
|---|---|---|
| [[ADR-0013-mail-provider]] | Hosted transactional mail provider | Accepted |
| [[ADR-0014-location-time-licence]] | Geofence plus optional event code | Accepted |
| [[ADR-0015-hosting-and-deployment]] | Local Docker, CI without deployment | Accepted |
| [[ADR-0016-supabase-plan-region]] | Supabase Free tier, Frankfurt | Accepted |
| [[ADR-0017-playback-session]] | PlaybackSession aggregate, client-driven advance | Accepted |
| [[ADR-0018-load-testing-tool]] | Gatling for load testing | Accepted |
| [[ADR-0019-hub-action-auditing]] | Audit hub actions with the same record | Accepted |
| [[ADR-0020-monorepo-layout]] | Backend and client in one repository | Accepted |
| [[ADR-0021-backend-dependency-set]] | MediatR, FluentValidation, built-in rate limiting and OpenAPI | Accepted |
| [[ADR-0022-observability-stack]] | Serilog with OpenTelemetry | Accepted |
| [[ADR-0023-catalog-cache]] | Two-level catalog cache | Accepted |
| [[ADR-0024-expiry-and-cleanup]] | Lazy expiry checks, one cleanup service | Accepted |
| [[ADR-0025-agent-git-protocol]] | Git protocol for unattended agent runs | Accepted |
| [[ADR-0026-environment-bring-up]] | Environment bring-up sequence | Accepted |

An accepted ADR is never edited. It is superseded by a new one.

---

## 16. Requirements traceability

Every mandatory requirement of R1, mapped to the design element that satisfies it. This is the table to walk through at defence.

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.1 | email+password or social registration | §10.1, §10.2, §13.1 | **partly implemented** – `POST /api/v1/auth/registrations` persists an unverified `User` with a BCrypt hash at work factor >= 12, `ux_users_email` arbitrates the race, and both a fresh and an already-registered address answer `202` so the endpoint is not an enumeration oracle. Social registration is designed only. A verified account can now sign in: `POST /api/v1/auth/sessions` verifies the stored hash and answers a 15-minute JWT plus an opaque refresh token, and every failure – unknown address, wrong password, unverified account – answers the same bodiless `401` for the same amount of hashing work. Rate limiting is **in place** on this endpoint and on every other credential endpoint — per-IP policy plus a per-account counter, §13.3 — so T-25 is closed |
| V.1 | link a social account | §10.2, Identity context | designed |

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.1 | email verification + forgot password | §10.1, §13.1 | **partly implemented** – verification works end to end: registration issues a 256-bit token, stores only its SHA-256 hash and hands the plaintext to `IMailService`; `POST /api/v1/auth/email-verifications` spends it exactly once through a `consumed_at_utc IS NULL AND expires_at_utc > @now` conditional update and sets `is_email_verified`. `204` verified, `410` expired, `400` for unknown, malformed and already-consumed alike, so the endpoint is not an enumeration oracle. A parallel test proves 20 concurrent verifications of one token produce exactly one `204`. Resending works too: `POST /api/v1/auth/verification-requests` issues a fresh token for an existing unverified account and supersedes that account's outstanding tokens in the same transaction, so exactly one link is live at a time; an unknown address, an already-verified account and a failed send are all answered `202` and send nothing, so the endpoint is not an enumeration oracle. The only `IMailService` today is the Development-only `LoggingMailService`, which writes the link to the log and makes any other environment fail at start-up – the hosted provider is O17, so no mail reaches a real mailbox yet (T-26). **Forgot password works end to end:** `POST /api/v1/auth/password-reset-request` answers 202 for any |

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.1 | three profile visibility tiers | §6.2, §13.2 | designed |
| V.2.1 | live vote-ordered queue | §5.2, §10.3, §9.2 | designed |
| V.2.1 | a highly-voted track **plays** earlier | §10.6, §11.2, [[ADR-0017-playback-session]] | designed |
| V.2.1 | public/private visibility | §6.2, §13.2 | designed |
| V.2.1 | licence modes incl. location+time | §6.2, [[ADR-0014-location-time-licence]] | **designed** — geofence + event code |
| V.2.1 | vote race conditions | §12.1 #1, §12.2 | designed |
| V.2.2 | per-device control licence, delegation | §5.2, §10.5, §11.3 | designed |
| V.2.2 | what a delegate actually controls | `PlaybackSession.controllerUserId` – [[ADR-0017-playback-session]] | designed |
| V.2.3 | realtime multi-user playlist editing | §9.2, §10.4 | designed |
| V.2.3 | reorder race conditions | §12.1 #2, §12.2 | designed |
| V.2 | at least 2 of the 3 services | §5.2, [[ADR-0010-service-scope]] | **designed** — all three |
| V.3 | backend is the single source of truth | §4.2, §8.1 | designed |
| V.4 | REST, JSON, reference documentation | §9.1, [[endpoint-inventory]], [[ADR-0021-backend-dependency-set]] | **partly implemented** – `docs/api/openapi.json` is generated from the code and committed; it documents nine paths — the two health endpoints and the seven identity endpoints that exist: `POST /api/v1/auth/registrations`, `/auth/email-verifications`, `/auth/verification-requests`, `/auth/sessions`, `/auth/tokens`, `/auth/password-reset-requests` and `/auth/password-resets`. Each mutating route documents the `400` its mandatory client headers can produce |
| V.5 | client is a remote control | §4.1, §9.1 | designed |
| V.5 | configurable backend address | §9.1, [[musicroom-mobile]] | designed |
| V.5 | social authentication on mobile | §10.2 | designed |
| V.6 | own data only, never another user's | §13.2 | designed |

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.6 | brute force and session theft protections | §13.1, §13.3 | **implemented** – **brute force:** every credential endpoint carries a per-IP policy (one per endpoint, so a registration loop cannot spend the sign-in allowance) and a per-account counter with progressive backoff, answering `429` with `Retry-After` and an empty body; the numbers are in §13.3 and the trip is logged without naming the account. **Session theft:** refresh tokens are stored only as SHA-256 digests, rotate on every use through a `consumed_at_utc IS NULL AND revoked_at_utc IS NULL AND expires_at_utc > @now` conditional update, and presenting an already-rotated token revokes the whole family in the same transaction. Parallel tests prove both concurrency claims: 20 concurrent rotations of one token produce exactly one `200`, one spent row and one replacement. Remaining gap, recorded and not silent: limiter state is per process, so several instances would each hold their own – O8 |
| V.6 | document remaining threats | §13.6, [[threat-register]] | **designed** — register written, verification pending |

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.6 | log platform, device, app version per action | §10.7, §13.4, [[ADR-0019-hub-action-auditing]] | **partly implemented – REST is done:** middleware refuses any mutating request that omits `X-Client-Platform`, `X-Client-Device` or `X-Client-App-Version` with a bodiless `400`, and an `AuditBehaviour` writes exactly one `audit_records` row per handled command, carrying those three values plus the action, the target type, the trace id and the instant. The record is written **inside the command's transaction**, so an effect without a record cannot exist and a record without an effect cannot either; if the audit write fails the whole command fails (owner decision, 2026-09-27, fail closed). A command refused by validation leaves no record. `user_id` is null on every row until an endpoint authenticates its caller, because every mutating endpoint that exists is anonymous – stated, not hidden. **The hub filter is still designed only:** no hub exists yet, so ADR-0019's filter arrives with the first one rather than being registered against nothing |
| V.7 | measure and justify concurrent users | §14.4, [[ADR-0018-load-testing-tool]] | tool decided; measurement not started |
| V.7 | declare server specification | §14.1, [[ADR-0015-hosting-and-deployment]], [[ADR-0016-supabase-plan-region]] | machine and region decided; measurement not started |
| V.6 | OWASP conformance of all code | [[asvs-conformance]], §13 | target committed, evidence pending |

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.8 | per-layer tests | [[testing]], [[ADR-0012-test-stack]] | **partly implemented** – 128 tests across four projects: validation and pipeline behaviours (`MusicRoom.Application.Tests`), infrastructure (`MusicRoom.Infrastructure.Tests`), persistence (`MusicRoom.Persistence.Tests`, which holds the built model to the schema naming conventions without needing an engine) and API integration (`MusicRoom.WebAPI.Tests`, Testcontainers-backed against a real PostgreSQL). Includes the four concurrency tests AGENTS.md requires, each asserting final database state: 20 parallel registrations of one address create exactly one row; 20 parallel verifications of one token produce exactly one `204` and one `consumed_at_utc`; 20 parallel rotations of one refresh token produce exactly one `200`, one spent row and one replacement; and 20 parallel resets of one token produce exactly one `204` and no live refresh token. Domain and realtime layers arrive with the slices that create them |
| V.8 | continuous integration | [[ci-cd-containerization]] | **done** – `.github/workflows/backend-dev.yml` and `backend-prod.yml`: restore, format, build -warnaserror, test, vulnerable-package scan, OpenAPI drift check; prod also builds the image. No deployment, by [[ADR-0015-hosting-and-deployment]] |

| Subject | Requirement | Design element | Status |
|---|---|---|---|
| V.8 | credentials in a git-ignored `.env` | §14.3, `.env.example` | **done** |
| V.8 | enforced code quality | [[dotnet-csharp]], `.editorconfig`, `MusicRoom/SonarLint.xml` | **done** – proven live in S0: a 47-line method fails the build with `S138 ... greater than the 40 lines authorized`, and six parameters fail with `S107`. The thresholds moved to `SonarLint.xml`; in `.editorconfig` they were silently ignored |
| IV.1 | no third-party source in the repo | §5.1, `.gitignore` excludes `reference/` | **done** |
| IV.1 | dependencies install from a fresh clone | §14.2, `Makefile`, `docker-compose.yml` | **done** – clone, `cp .env.example .env`, fill, `make up`. Verified end to end: the API container serves `/health` and `/health/ready` against the compose PostgreSQL |
| IV.2 | mobile app covers all operations | `MusicRoomApp/`, [[musicroom-mobile]] | sequential workstream, owned by the UI developer; starts after S9 — [[delivery-plan]] "Work split" |
| VII | submitted in the git repository | [[ADR-0020-monorepo-layout]] | **designed** – one clone holds backend, client and documentation |
| III | the SDK must not do our work | §4.2, §9.3 | designed |

"Designed" means the decision and the mechanism are recorded, and nothing more. "Partly implemented" and "done" name code that exists on a branch and passes `make agent-check`; the row says which part, because a requirement is rarely finished in one slice.

---

## 17. Open design issues

Closed rows stay visible: a decision log that hides what it used to not know is less useful at defence, not more.

**Open**

| # | Issue | Blocks | Note |
|---|---|---|---|
| O8 | SignalR scale-out backplane (Redis) | multi-instance deployment | **deliberately open.** Not needed for a single-instance defence |

| # | Issue | Blocks | Note |
|---|---|---|---|
| O11 | remote deployment target | a live demo URL | [[ADR-0015-hosting-and-deployment]]. Fly.io, Railway, Render, Azure Container Apps. **Not Vercel** |
| O12 | fluent assertion library | test readability | add only if xUnit's built-ins prove insufficient |
| O13 | silent-controller recovery | a stalled playback session | [[ADR-0017-playback-session]] accepted no watchdog; takeover is the manual path. Revisit only if it bites in testing |
| O18 | self-hosted Infisical vault | API keys sitting readable in `.env` | [[ADR-0029-containerised-dev-and-secret-vault]], amended 2026-09-27: **required scope, not optional.** The provider abstraction lands early, the vault at S1b, after the mail adapter at S1a (O17). Self-hosted Infisical needs its own PostgreSQL, Redis and encryption key — roughly five compose services instead of two. Subject V.8 does not ask for it; the owner does, and the delivered system runs on it. It is planned alongside S1 to S9, not after them |
| O17 | real mail provider adapter | V.1 email verification actually reaching a mailbox | owner decision 2026-09-24: SPEC-003 ships a Development-only logging `IMailService`; the hosted-provider adapter is a separate spec at the end of S1. A logging implementation cannot satisfy V.1 at the defence |
| O14 | music preferences have no consumer | nothing | V.1 requires storing and updating them, which is met. Note for defence: storage-only today |

| # | Issue | Blocks | Note |
|---|---|---|---|
| O19 | RS256 key pair: generation, storage, JWKS exposure and rotation with an overlap window | the OpenID Provider surface at S8 | [[ADR-0031-musicroom-as-openid-provider]]. `EnvSecretProvider` holds the private key until the vault lands at S1b, after which it belongs there. Rotation needs an overlap so tokens signed by the outgoing key keep verifying |
| O16 | OpenTelemetry must earn its place | credibility of the V.7 bottleneck claim | [[ADR-0022-observability-stack]]: capture one trace during the Gatling run showing EF Core, HttpClient and total request time together |

**Closed**

| # | Issue | Closed by |
|---|---|---|
| O1 | which services ship | [[ADR-0010-service-scope]] — all three |
| O9 | concrete table and column definitions | the migrations, not a document: `InitialIdentity`, `RefreshTokens`, `PasswordResetTokens` and `AuditRecords` ([[migrations-index]]). `SchemaConventionTests` holds the built model to the naming rules |
| O15 | MediatR licence terms | [[ADR-0021-backend-dependency-set]] records the verification of 2026-09-24: the free Community License covers this project, so MediatR stays and the hand-written dispatcher is not needed |
| O10 | `Makefile` and `docker compose` | slice S0 — both exist, `make up` verified end to end against the compose PostgreSQL |
| O2 | solution and folder layout | [[ADR-0011-solution-layout]] |
| O3 | test framework | [[ADR-0012-test-stack]] |
| O4 | API hosting target | [[ADR-0015-hosting-and-deployment]] |
| O5 | Supabase plan tier and region | [[ADR-0016-supabase-plan-region]] |
| O6 | SMTP provider | [[ADR-0013-mail-provider]] |

| # | Issue | Closed by |
|---|---|---|
| O7 | location+time licence enforcement | [[ADR-0014-location-time-licence]] |
| G1 | queue advance / playback progression | [[ADR-0017-playback-session]] |
| G2 | what a control delegate actually controls | [[ADR-0017-playback-session]] |
| G3 | load testing tool | [[ADR-0018-load-testing-tool]] |
| G4 | hub actions missing from the audit log | [[ADR-0019-hub-action-auditing]] |

---

## Related

[[HOME]] · [[MEMORY]] · [[system-overview]] · [[domain-model]] · [[realtime-architecture]] · [[auth-and-identity]] · [[decisions-index]] · [[subject-requirements]] · [[technical-guide-index]] · [[user-manual]]